

nodes with specific configurations to test that your whole ROS program is working properly inside the ROS framework by executing unit tests from within.

That is why, for the creation of tests for those two levels, we need to combine `code>rostd` with either `code>gtest` (C++ case) or `unittest` (Python case). You can get a full documentation about using `code>rostd` at this link.



Creating tests for Level 4

Level 4 tests allow us to see if the programme is actually doing what it should.

We have three possibilities for testing this, ranging from the most remote to the most realistic.

Using mocks for testing

If you're a developer, you're already familiar with mocks. You can make your own mocks to simulate your robot's connection to various portions of the ROS API. On this page, you'll learn how to use mocks in ROS. Working with mocks in ROS is a time-consuming process that necessitates extensive planning. It's also of limited utility because it can only create what you've already put on it. When it comes to development, it is not recommended to use mocks.

Only use it if none of the other alternatives is available or if you're writing unit tests for your code. You can try out any of the following methods for writing unit tests for your code. In any case, if you wish to test with mocks, try using the Google Test environment: <https://github.com/google/googletest>.

Using ROS bags for testing

The whole ROS API of a robot can be recorded in a log file while it is operating in a real-world environment, and then run it on another machine afterwards. A ROS bag is the name for this log file. When you run a ROS bag on another machine, it will display the same ROS API that it did when it was recorded.